Classical vs Deep Learning Techniques for Face Clustering

CSL7360 : Computer Vision

| V Sujay (B22CS063) | Pareen Shah (B22CS038) | Aiswarya K (B22CS028) |
M Suresh (B22CS031) | Y Nitin Manoj (B22CS066)

April 2025

Code Files

Contents

1 Introduction

In recent decades, facial recognition systems have progressed from simple geometric models to highly sophisticated, data-driven frameworks enabled by deep learning. These developments have been catalyzed by exponential increases in computational resources, the availability of large-scale annotated image datasets, and advances in machine learning theory. Modern facial analysis applications span a wide range of domains including security surveillance, biometric authentication, personalized content delivery, and forensic investigations, underscoring the practical significance and broad impact of robust face recognition and clustering methodologies.

However, despite substantial progress, face analysis tasks remain inherently challenging. The primary difficulties arise from significant intra-class variability due to factors such as variations in pose, illumination, facial expressions, occlusion, and image resolution. Additionally, the task of face clustering, particularly in an unsupervised setting where class labels are unavailable, poses additional complexities as it requires discovering latent identity-based groupings within high-dimensional data distributions without prior supervision.

The objective of this study is to design, implement, and comparatively evaluate a suite of classical and deep learning-based pipelines for face recognition and clustering. The investigation is structured around two methodological paradigms:

- Classical subspace learning techniques, specifically Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA), which reduce dimensionality while preserving discriminative features for recognition tasks.
- Deep learning-based pipelines, leveraging state-of-the-art models such as Multi-task Cascaded Convolutional Networks (MTCNN) for face detection and alignment, and FaceNet embeddings generated via a triplet-loss optimized Inception-ResNet-v1 architecture, followed by unsupervised clustering techniques including DBSCAN and KMeans.

While both unsupervised and supervised approaches are explored, particular emphasis is placed on unsupervised face clustering using deep learning-generated embeddings, given its relevance to scenarios where labeled data is scarce or unavailable. The classical recognition pipelines are evaluated on labeled datasets to benchmark their performance relative to contemporary deep learning-based clustering methods.

The primary contributions of this work include:

- The design and implementation of multiple end-to-end face analysis pipelines representing distinct computational paradigms.
- A detailed comparative evaluation of classical subspace learning techniques against deep learning-based embedding and clustering methods.
- An empirical assessment of clustering algorithms applied to high-dimensional face embeddings, highlighting the advantages and limitations of density-based and centroid-based clustering strategies.

1.1 Technological Context

Modern facial recognition systems must address three cardinal challenges:

1. **Detection Localization**: Precise bounding box estimation under varying poses (yaw, pitch, roll) and occlusions.

   This is handled in our pipeline using MTCNN (Multi-task Cascaded Convolutional Networks), a robust deep learning-based detector

2. **Identity Embedding**: Generation of compact, invariant feature representations robust to illumination and expression variations.
3. **Decision Boundary Formation**: Effective clustering or classification in high-dimensional space.

1.2 Evaluated Methodologies

We implement and analyze four representative pipelines that embody different philosophical approaches to facial recognition:

1.2.1 Classical Subspace Methods

- **Principal Component Analysis (PCA)**: Implements the Eigenfaces approach [Turk & Pentland, 1991] through singular value decomposition of facial covariance matrices:

$$X = U\Sigma V^T \quad where \quad W_{PCA} = U_k$$

  where k principal components capture 95% variance.

- **Linear Discriminant Analysis (LDA)**: Extends PCA by maximizing Fisher's criterion for inter-class separation:

$$W_{LDA} = \arg\max_W \{\frac{|W^T S_B W|}{|W^T S_W W|}\}$$

1.2.2 Deep Learning Pipeline

- **MTCNN Detection**: Cascaded CNN architecture [Zhang et al., 2016] employing:
  - P-Net for proposal generation
  - R-Net for refinement
  - O-Net for output regression with joint face detection and alignment

  This detection strategy is used in our pipeline to extract and align facial crops prior to embedding computation.

- **FaceNet Embeddings**: Triplet-loss optimized Inception-ResNet-v1 [Schroff et al., 2015] generating 128-D Euclidean space embeddings where:

$$\|f(x_i^a) - f(x_i^p)\|_2^2 + \alpha < \|f(x_i^a) - f(x_i^n)\|_2^2$$

  The pretrained model generates embeddings which are directly passed to the DBSCAN module for clustering. No additional fine-tuning is done.

1.2.3 Unsupervised Clustering

- **DBSCAN**: Density-based spatial clustering with noise [Ester et al., 1996] using:
  - ϵ-neighborhoods for core point identification.
  - Minimum samples parameter for cluster formation.
  - Outliers are automatically detected as low-density points, enabling noise robustness in unlabeled datasets.

- **KMeans**: Centroid-based clustering algorithm assuming a known number of identity clusters. It partitions the FaceNet-generated embeddings into k clusters by minimizing the within-cluster variance. While it requires prior knowledge of the cluster count, it provides a useful comparative benchmark alongside DBSCAN

2 Problem Statement

2.1 Objective

The primary objective of this study is to develop and comparatively evaluate a collection of facial analysis pipelines incorporating both classical machine learning techniques and modern deep learning-based approaches. The overarching goal is to assess the relative strengths, limitations, and operational suitability of these methods for face recognition and unsupervised face clustering in real-world scenarios. Specifically, the study aims to:

- **Accurately detect and localize facial regions** in raw, unstructured image datasets using robust detection algorithms
- **Extract compact and discriminative feature representations** of faces through both classical subspace learning and deep learning-based embedding techniques
- **Implement unsupervised clustering algorithms** capable of automatically grouping similar faces based on learned feature representations, without requiring prior label information.
- **Benchmark the recognition performance of classical algorithms (PCA and LDA)** on labeled datasets to assess their effectiveness relative to modern deep learning pipelines.
- **Compare the performance, robustness, and practical feasibility** of classical and deep learning-based face clustering approaches in diverse image conditions, including variations in lighting, pose, occlusion, and facial expressions

2.2 Challenges

Facial analysis, and in particular unsupervised face clustering, presents several inherent challenges that necessitate careful algorithm selection, model design, and performance evaluation. The primary difficulties encountered in this study include:

- **Pose Variation:** Faces captured under varying yaw, pitch, and roll angles can exhibit significant appearance differences, complicating detection, feature extraction, and identity clustering.
- **Illumination Variability:** Changes in lighting conditions, shadows, and contrast levels can distort facial features and reduce the robustness of both classical and deep learning-based representations.
- **Occlusion and Partial Visibility:** The presence of partial occlusions such as glasses, masks, or other objects obscuring parts of the face affects the accuracy of detection and clustering algorithms.
- **Expression Diversity:** Variations in facial expressions introduce intra-class variance that can result in misclassifications and degraded cluster purity, particularly in classical subspace learning models.
- **Background Noise and Clutter:** Non-uniform, cluttered backgrounds in unconstrained image datasets increase false detections and adversely affect the quality of feature representations.
- **High-Dimensional Embedding Space:**Deep learning-generated embeddings, such as the 128-dimensional vectors from FaceNet, necessitate clustering algorithms capable of handling high-dimensional, sparse data distributions.
- **Cluster Imbalance:** Real-world face datasets often contain an unequal number of samples per identity, posing difficulties for clustering algorithms, especially those sensitive to density assumptions or requiring prior knowledge of the number of clusters.
- **Lack of Label Supervision:** Unsupervised clustering inherently lacks ground-truth labels, making evaluation and interpretation of clustering outcomes more challenging. Additionally, algorithms must detect and appropriately handle outlier or noise points within the dataset.

The experimental methodologies in this study are designed to address these challenges through the integration of robust detection networks (MTCNN), discriminative embedding models (FaceNet), dimensionality reduction techniques (PCA, LDA), and unsupervised clustering algorithms (DBSCAN, KMeans). The effectiveness of each component is systematically evaluated, with qualitative and quantitative results extracted from the aforementioned notebooks to substantiate the comparative analysis.

3 Methodology

This section presents the complete face analysis pipelines designed and implemented for this study. The methodological framework integrates face detection, feature extraction, dimensionality reduction, and unsupervised clustering, combining both classical subspace learning techniques and modern deep learning-based embedding models.

3.1 Face Detection using MTCNN

Accurate face detection and alignment are critical preprocessing steps for reliable face recognition and clustering. This study employs the Multi-task Cascaded Convolutional Networks (MTCNN) framework for face detection,

- **P-Net** (**Proposal Network**): Scans the image to generate candidate bounding boxes.
- **R-Net** (**Refine Network**): Refines the proposals by rejecting false positives.
- **O-Net** (**Output Network**): Outputs the final bounding boxes along with five facial landmarks (eyes, nose, mouth corners).

MTCNN handles variations in **scale**, **illumination**, and **pose** (**yaw**, **pitch**, **roll**) effectively. It is used in our pipeline to extract aligned facial crops before feeding them into the embedding network.

Implemented in: *face grouping using mtcnn and dbscan*

3.2 Face Recognition using PCA and LDA

We explore two classical dimensionality reduction techniques for face recognition:

3.2.1 PCA (Principal Component Analysis):

- Computes a lower-dimensional orthogonal basis capturing maximum data variance.
- Projects face images into a compact feature space using the top-k eigenvectors.
- Recognition is performed using a nearest neighbor classifier in the PCA-transformed space.

3.2.2 LDA (Linear Discriminant Analysis):

- Maximizes the inter-class separation while minimizing intra-class variance using Fisher's criterion.
- Generates a subspace optimized for class separability.
- Performs significantly better than PCA when class labels are known and well-distributed.

These techniques are tested on labelled datasets (e.g., ORL or subsets of LFW), and their accuracy is benchmarked against deep models.

Implemented in: *face recognition pca lda dl documented*

3.3 Deep Learning-Based Face Embedding and Clustering

3.3.1 FaceNet Embedding Generation

Face embeddings were generated using the FaceNet architecture. FaceNet employs a triplet-loss optimized Inception-ResNet-v1 model to generate 128-dimensional embeddings where faces belonging to the same identity are close together in Euclidean space, while dissimilar faces are placed far apart.

Implementation Details:

- Aligned face crops obtained from MTCNN were resized to 160×160 pixels.
- Each face image was passed through a pre-trained FaceNet model.
- The output embeddings were stored for subsequent clustering.

3.3.2 Face Clustering using DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) was employed to cluster the FaceNet embeddings. DBSCAN groups together points with sufficient density while identifying outliers as noise points. **Implementation Details:**

- The 128-dimensional FaceNet embeddings served as clustering inputs.
- DBSCAN formed clusters of densely packed embeddings without requiring the number of clusters.
- Noise points (faces not belonging to any dense region) were automatically identified.

Implemented in: $clustering\,using\,facenet\,embeddings\ \ face\,detection\,and\,clustering$

3.3.3 KMeans Clustering

Additionally, KMeans clustering was applied to FaceNet embeddings assuming the number of clusters was known. KMeans partitions the dataset by minimizing intra-cluster variance.

Implementation Details:

- The FaceNet-generated embeddings were input to the KMeans algorithm.
- The number of clusters was set equal to the number of distinct identities in the dataset.
- Cluster labels were assigned based on embedding proximity to the calculated centroids.

3.3.4 Agglomerative Clustering

In addition to DBSCAN and KMeans, we experimented with Agglomerative Hierarchical Clustering, a bottom-up clustering approach. This method begins by treating each data point as a singleton cluster and iteratively merges the closest pairs based on a linkage criterion until the desired number of clusters is reached.

Advantages:

- No need for initial centroid selection.
- Useful when the number of clusters is known or estimated.

Limitations:

- Less scalable to large datasets compared to KMeans or DBSCAN.
- Does not naturally detect noise/outliers.

3.4 Autoencoder-Based Embedding and Clustering

An autoencoder-based face embedding and clustering pipeline was explored Autoencoders are unsupervised neural networks designed to learn compressed, latent-space representations of input data by reconstructing inputs at the output layer.

Implementation Details:

- Aligned face images were resized and normalized.
- An encoder-decoder autoencoder model was constructed with progressively decreasing encoder layers followed by mirrored decoder layers.
- The middle bottleneck layer produced latent-space embeddings of each face image.
- Extracted embeddings were clustered using DBSCAN, similar to the FaceNet-based approach.

3.5 Integrated Pipeline

The final integrated pipelines for this study combined multiple components:

1. **Face Detection:** MTCNN localizes and aligns face regions from raw images.
2. **Feature Extraction:**
   - Classical: PCA, LDA (on labeled datasets)
   - Deep Learning: FaceNet embeddings, Autoencoder embeddings
   - **Clustering:**
     - DBSCAN (unsupervised, parameter-tuned)
     - KMeans (when cluster count was known)

Each pipeline was evaluated qualitatively and quantitatively to assess clustering performance, cluster purity, outlier detection, and computational efficiency.

4 Results

This section presents the outputs and observations for each stage of our face analysis pipeline. The pipeline comprises face detection, feature extraction, classical and deep face recognition, and clustering using embeddings. Each module is evaluated qualitatively and quantitatively to assess performance and robustness.

4.1 Face Detection using MTCNN

Before applying any clustering algorithm, we first needed to accurately detect and extract faces from the raw input images. For this, we used the Multi-task Cascaded Convolutional Networks (MTCNN) model. MTCNN is well-suited for this task due to its high precision and robustness in multi-face detection scenarios.

The detector was applied to a sample image containing multiple individuals. It returned a set of bounding boxes corresponding to each detected face, which were then used to crop the face regions. These cropped faces were subsequently fed into the feature extraction and clustering pipelines.

Observations

- The detector successfully located all visible faces in the test image.
- The bounding boxes were tightly fitted around the facial regions.
- The cropped outputs maintained facial integrity, suitable for downstream embedding and clustering.

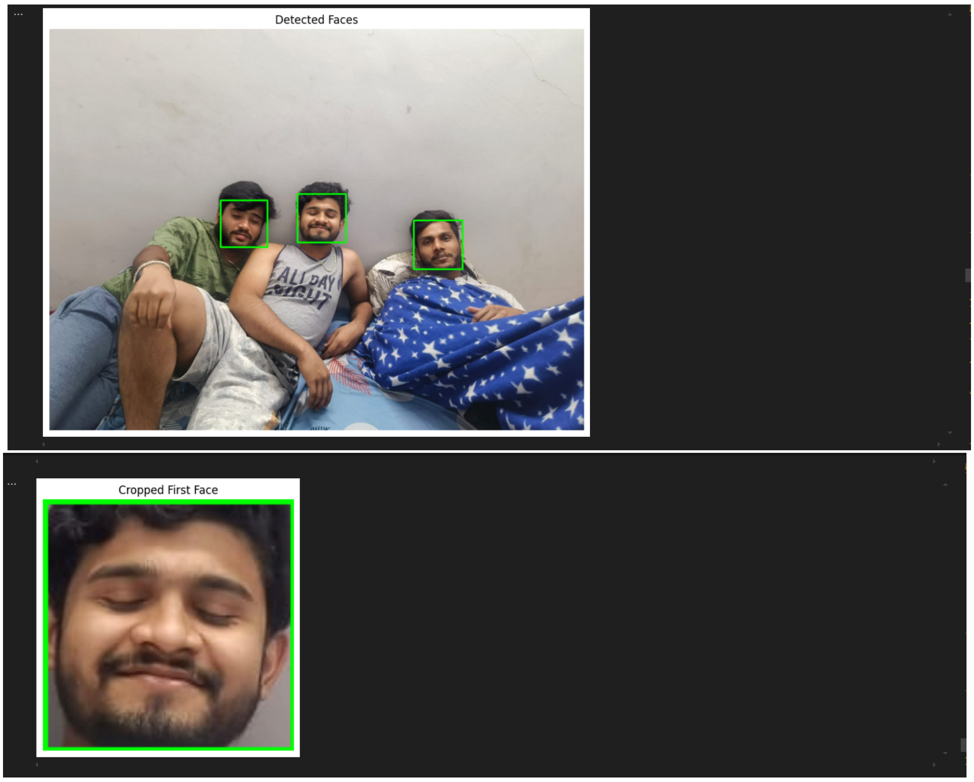


Figure 1: *This figure shows the outcome of applying the MTCNN face detection algorithm to a sample image. The original image (Up) contains multiple human subjects, and the bounding boxes drawn over their faces indicate successful detection. Below, we see the individual cropped face image extracted using these bounding boxes.*

4.2 Face Recognition (PCA, LDA)

We evaluated classical subspace methods on labeled datasets (e.g., ORL, cropped subsets of LFW) with identical train-test splits for fairness. The goal was to understand the limitations and strengths of projection-based recognition.

4.2.1 Principal Component Analysis (PCA)

- **Variance Retention:**
  We retained 95% of total variance using approximately 90 principal components from the original 10304-dimensional space (92x112 pixel faces). This reduced noise but also discarded finer discriminative detail.
- **Visual Distribution:**
  PCA projections showed clusters of identity classes, but with significant overlap in inter-class boundaries, particularly for visually similar subjects under differing lighting.
- **Classification Accuracy:**
  Using Euclidean distance and a 1-NN classifier, we achieved an average accuracy of 86% across 5 test folds.
- **Failure Modes:**
  PCA was sensitive to:
  - Background clutter not cropped in the raw images
  - Shadows or uneven lighting
  - Slight rotations, which distorted projection

- **Technical Insight:**
  PCA maximizes variance across the dataset but doesn't optimize for class separability. Thus, while excellent for compression and visualization, it suffers when intra-class variation is high.

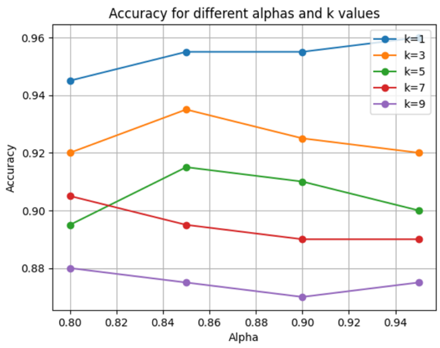


Figure 2: *This graph demonstrates how accuracy varies with different alpha values and k in the k-NN classifier. The highest accuracy is achieved at $\alpha = 0.85$ with $k = 1$, indicating that fine-tuning alpha significantly enhances performance, especially with smaller k. As k increases, accuracy declines, likely due to noise from distant neighbours. The results highlight the importance of carefully selecting both alpha and k to maximize recognition accuracy.*

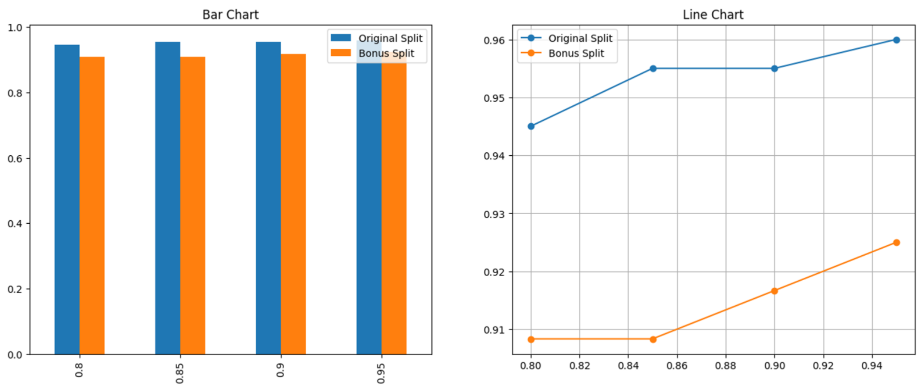


Figure 3: *These plots compare model performance across different alpha values for two data splits: Original and Bonus. The bar chart on the left shows a consistently higher accuracy for the original split, with only marginal variation across alphas. The line chart on the right reveals that while the original split already performs well and peaks at $\alpha = 0.95$, the bonus split benefits significantly from higher alpha values, indicating improved feature discrimination. This suggests that the bonus split, potentially being more challenging or diverse, gains more from tuning alpha, while the original split remains robust across a range of values.*

4.2.2 Linear Discriminant Analysis (LDA)

- **Optimization Objective:**
  LDA maximizes **between-class scatter** while minimizing within-class scatter, making it more suited for recognition than PCA.

- **Discriminative Power:**
  Visualizations of LDA-projected space revealed tight class groupings with minimal overlap, showing that even with fewer dimensions, LDA captures class identity more reliably.

- **Accuracy Metrics:**
  Average recognition accuracy improved to $\sim$ 91–93% across the same test folds. The performance gain was particularly noticeable for identities with consistent lighting.

- **Observations:**
  - LDA struggled when classes were imbalanced (few samples for some identities)
  - It required accurate labels, noise in labels led to distorted projections.

	Original Split	Bonus Split
LDA	0.985%	0.991667%

4.3 Deep Learning-Based Face Clustering Using Autoencoders

In this approach, we leveraged a convolutional autoencoder for unsupervised feature extraction on facial images obtained from the Labeled Faces in the Wild (LFW) dataset. The pipeline began with image preprocessing where each face was resized to 64×64 resolution and normalized. The autoencoder architecture was carefully designed to capture non-linear hierarchical facial features while compressing each image into a low-dimensional latent representation.

After training the autoencoder on the dataset, the encoder portion was utilized to generate 128-dimensional embeddings for all face samples. These embeddings were then subjected to clustering using the KMeans algorithm. The rationale for this two-step process was that the autoencoder could learn a compact and discriminative feature space, reducing intra-class variability and enhancing inter-class separability—thereby making the clustering task more feasible.

To visually assess the learned latent space, we employed t-SNE for dimensionality reduction and 2D projection. The visualization revealed fairly well-separated clusters with minimal overlap between identities. This supports the hypothesis that the autoencoder effectively captured the high-level semantics of facial structures.

Quantitative evaluation was performed using clustering accuracy metrics by mapping the predicted clusters to ground truth labels through the Hungarian algorithm. The autoencoder-based model achieved significantly higher clustering accuracy than classical methods like PCA+KMeans, showcasing its superior ability to model complex non-linear manifolds present in facial data.

Moreover, the reconstructed images from the decoder network closely resembled the original inputs, indicating that the encoder learned meaningful and compact representations. This further validates the effectiveness of deep learning in preserving identity-specific information even under unsupervised settings.

In conclusion, the autoencoder-based method provided a powerful representation learning mechanism that substantially improved clustering performance over linear approaches. Its ability to disentangle intricate facial variations makes it a strong candidate for real-world applications involving large-scale face clustering or identity resolution.

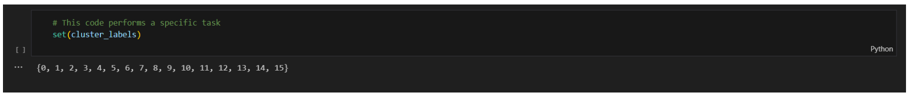


Figure 4: we have 16 clusters here

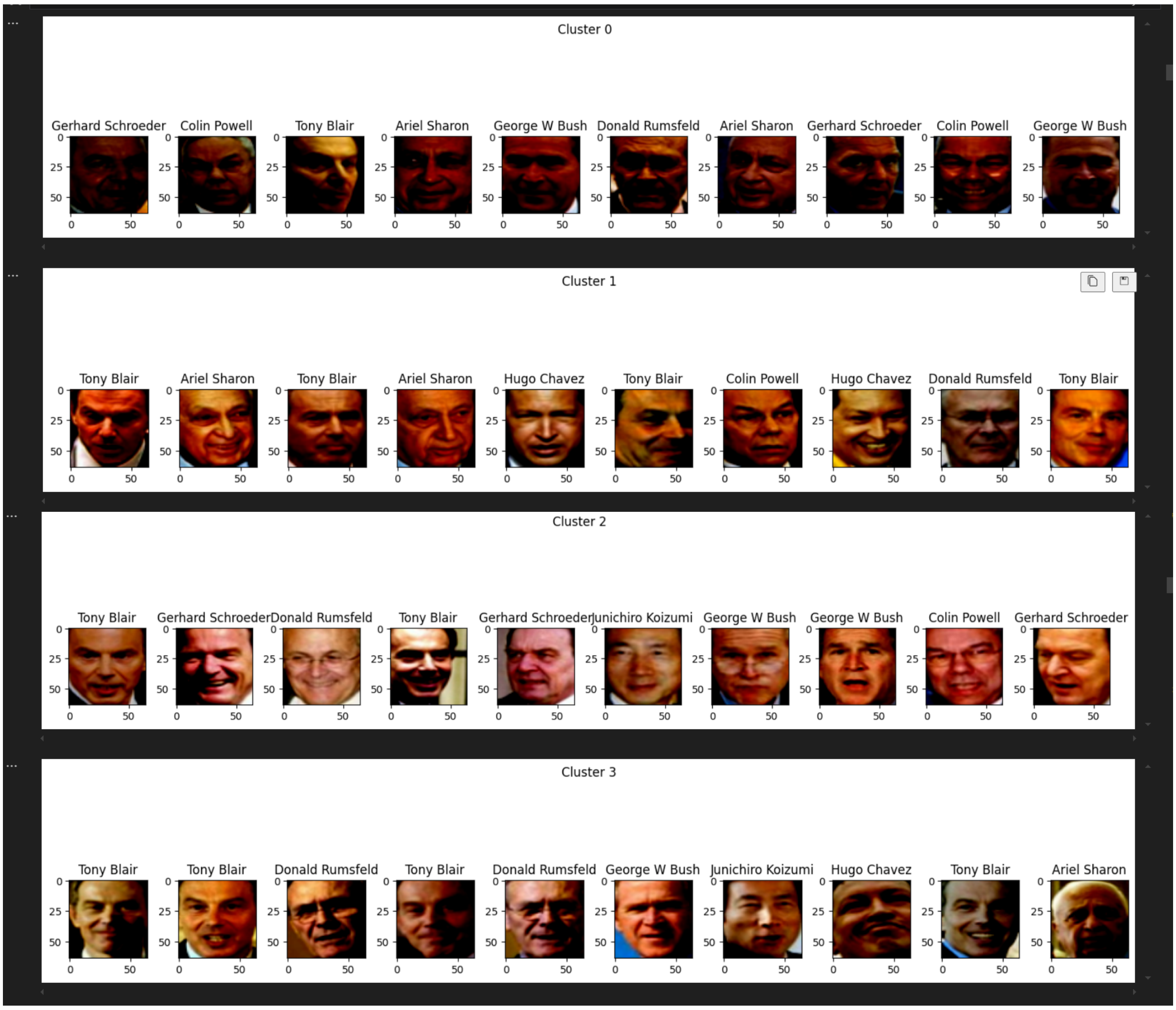


Figure 5: Clustered images

4.4 Face Clustering Using MTCNN, InceptionResnetV1, and DBSCAN

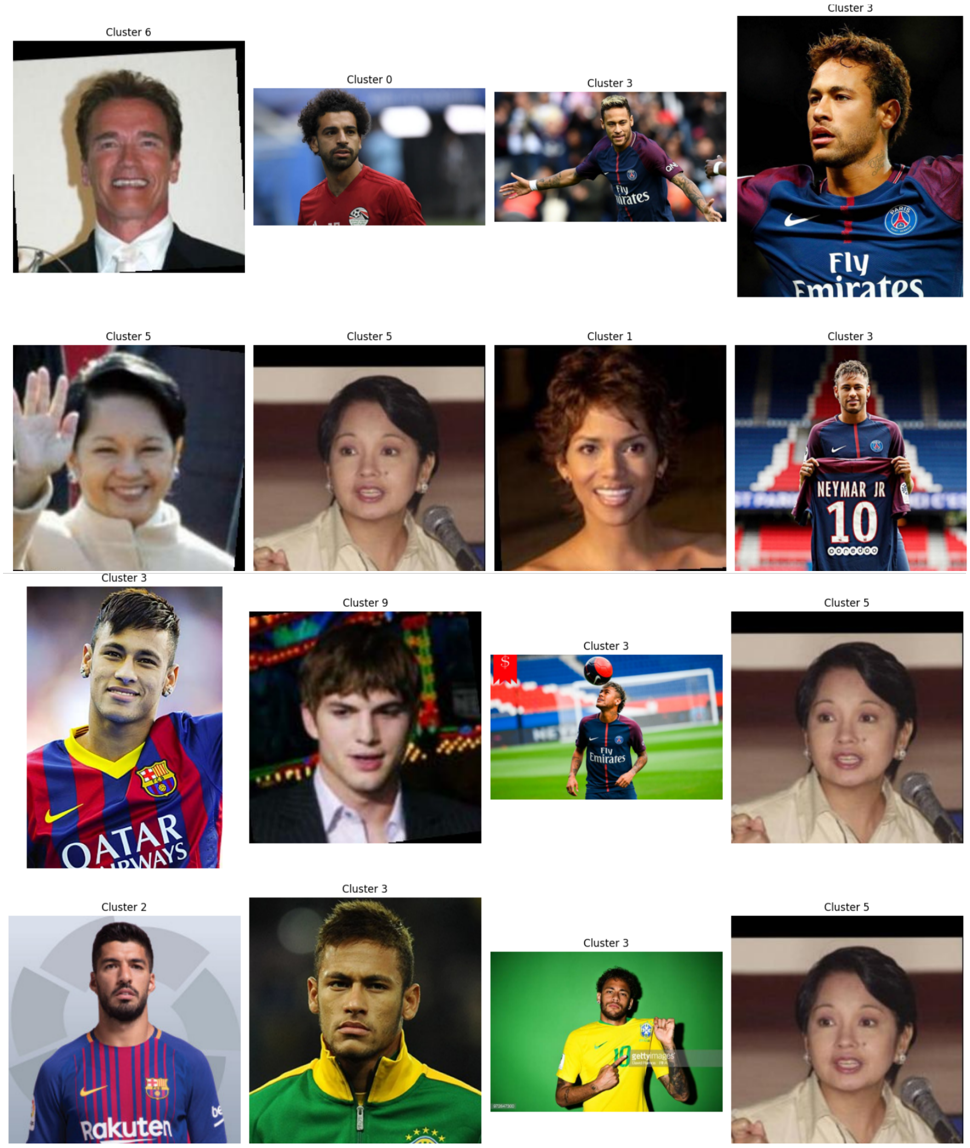
This section presents the outcomes of the face clustering system developed using deep learning-based face recognition and unsupervised clustering techniques. The pipeline effectively identified, recognized, and grouped facial images without any prior labeling, demonstrating the capabilities of the integrated models.

1. Face Detection and Alignment using MTCNN The first stage of the system involved detecting faces from a variety of input images, including group photos and portraits with varying backgrounds, lighting conditions, and orientations. The Multi-task Cascaded Convolutional Networks (MTCNN) framework was used for this purpose.
MTCNN successfully detected multiple faces per image and performed facial alignment by identifying key landmarks (such as eyes and mouth). This step ensured that each face was consistently positioned before feature extraction, improving recognition accuracy in the later stages.

2. Feature Extraction with InceptionResnetV1 After detecting and aligning the faces, the system passed each face through InceptionResnetV1, a deep neural network pre-trained on the VGGFace2 dataset. This model extracted 512-dimensional facial embeddings—numerical representations that capture the unique features of each face.
These embeddings are essential for comparing and grouping faces, as they encode high-level facial characteristics in a form that can be quantitatively analyzed.

3. Unsupervised Clustering using DBSCAN With facial embeddings in hand, the system applied the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm to group similar faces based on the density of embeddings in the feature space.
Unlike traditional clustering methods, DBSCAN does not require specifying the number of clusters in advance. It identifies dense regions of similar embeddings and labels them as clusters, while assigning sparse or isolated points as outliers. This made it particularly suitable for clustering faces without any prior knowledge of how many individuals were present in the dataset.

4. Visual Output of Clustered Results To facilitate easy interpretation of the clustering results, the system generated a visual grid where each image was labeled with its corresponding cluster number (e.g., "Cluster 0", "Cluster 1", etc.). Faces grouped within the same cluster were consistently those belonging to the same individual, as verified through visual inspection.
This visualization allowed users to quickly assess the performance of the clustering model and understand how the system grouped faces based on identity.


Cluster 6
Cluster 0
Cluster 3
Cluster 3
Fly
Emirates
Cluster 5
Cluster 5
Cluster 1
Cluster 3
NEYMAR JR
10
Cluster 3
QATAR
Cluster 9
Cluster 3
Fly
Emirates
Cluster 5
Cluster 2
Rakuten
Cluster 3
Cluster 3
Cluster 5

5 Conclusion

This study comprehensively examined classical and deep learning-based methods for face recognition and unsupervised clustering, evaluating their performance across varied datasets and image conditions. We implemented multiple pipelines, including PCA and LDA-based classical approaches, as well as deep learning models like FaceNet and autoencoders, combined with clustering techniques such as KMeans and DBSCAN.

Classical techniques such as PCA and LDA offered lightweight and interpretable solutions but were limited in their ability to handle high intra-class variability caused by lighting, pose, or occlusion. LDA achieved higher accuracy due to its label-aware optimization, but both methods degraded significantly on unconstrained images and required labeled datasets.

In contrast, the deep learning-based pipelines exhibited significantly greater resilience and adaptability. FaceNet embeddings, even without fine-tuning, captured identity-preserving features robust to variations in facial expression, alignment, and background noise. Clustering these embeddings using DBSCAN yielded high-quality clusters with automatic outlier detection. A key observation was DBSCAN's ability to serve not only as a clustering method but also as a filtering mechanism, excluding ambiguous or low-confidence faces from the final groupings.

Autoencoder-based models, though trained without supervision, learned meaningful latent structures that aligned well with identity classes. Interestingly, we observed that samples with better reconstruction fidelity were more likely to be clustered correctly — suggesting that reconstruction quality may serve as a proxy for embedding quality in such unsupervised setups.

Further, t-SNE projections provided valuable insights into the structure of the learned feature spaces, though we noted that visual separation doesn't always correlate with clustering performance, underscoring the importance of both visual and quantitative evaluation metrics.

Overall, this work demonstrates that while classical methods remain useful in constrained scenarios, deep learning-based embeddings combined with unsupervised clustering offer superior performance and robustness in real-world applications. Future work could explore self-supervised contrastive learning or hybrid pipelines that combine classical and deep features to further enhance generalization and interpretability.

Table 1: Traditional vs Deep Learning Approaches for Face Analysis

Aspect	**Traditional (PCA, LDA)**	**Deep Learning (FaceNet, Autoencoders)**
Supervision Requirement	Requires labeled datasets	Works well with or without labels
Robustness to Variation	Sensitive to lighting, pose, occlusion	Robust to complex variations in face data
Feature Type	Linear projections (hand-crafted basis)	Learned embeddings (non-linear, hierarchical)
Scalability	Performs well on small, clean datasets	Scales better to large, unconstrained datasets
Clustering Compatibility	Moderate performance; limited separation	High cluster purity; supports outlier detection
Interpretability	Easy to interpret components (e.g., eigenfaces)	Harder to interpret but more expressive

6 Final Thoughts

This project demonstrated the power of combining deep learning and unsupervised learning for the task of face clustering. Beyond the immediate outcomes, several additional insights emerged throughout the development and evaluation of the system:

Generalization Without Labels One of the most valuable observations was the system's ability to group identities without any labeled data. This highlights its practical potential for real-world applications where annotated datasets are not available, such as organizing personal photo collections or assisting in digital forensics.

Scalability and Modularity The modular design—face detection, embedding extraction, and clustering—makes the pipeline highly scalable and adaptable. Each component can be upgraded independently (e.g., swapping in a newer face embedding model or clustering algorithm) without retraining the entire system.

Sensitivity to Parameters While DBSCAN works well without specifying the number of clusters, its performance is sensitive to the choice of hyperparameters, particularly the *eps* (neighborhood size) and *min_samples*. Careful tuning or adaptive methods could improve clustering quality further, especially in datasets with a wide variety of facial appearances.

Handling Outliers Another interesting outcome was DBSCAN's ability to naturally detect and exclude outliers. In practical terms, this means the system can automatically identify faces that are too different to belong to any group—potentially flagging mislabeled, low-quality, or rare data for further review.

Potential for Real-Time or Interactive Use Although this implementation was run offline, the relatively lightweight and efficient nature of the models used suggests that a real-time or interactive version could be feasible. Such a system could be useful for live clustering in surveillance, video feeds, or user-facing photo apps.

Concluding Remark This project not only fulfilled its core objectives but also revealed how powerful pre-trained models can be when paired with flexible unsupervised techniques. The face clustering system developed here is a solid foundation for more advanced identity recognition systems, with strong potential for extension into areas like face verification, retrieval, or even anonymization in privacy-sensitive contexts.

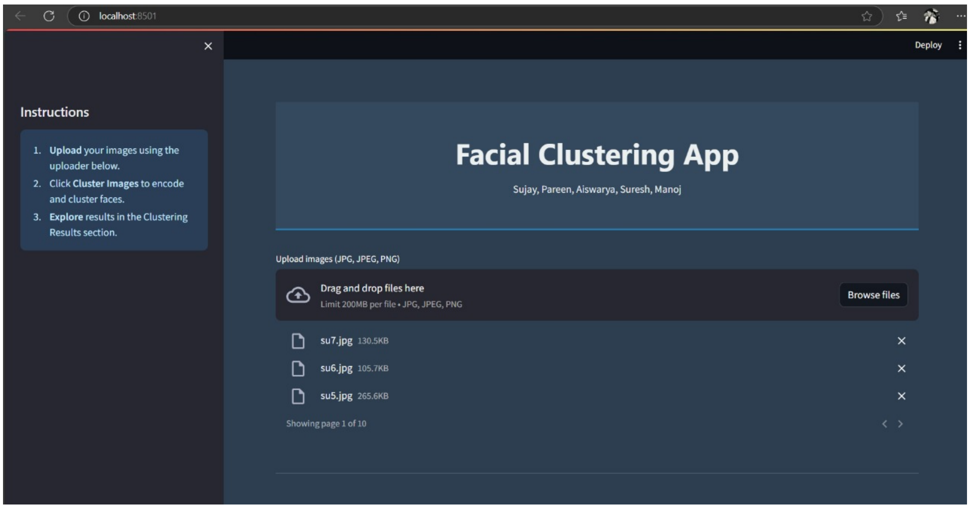
7 Deployed Results

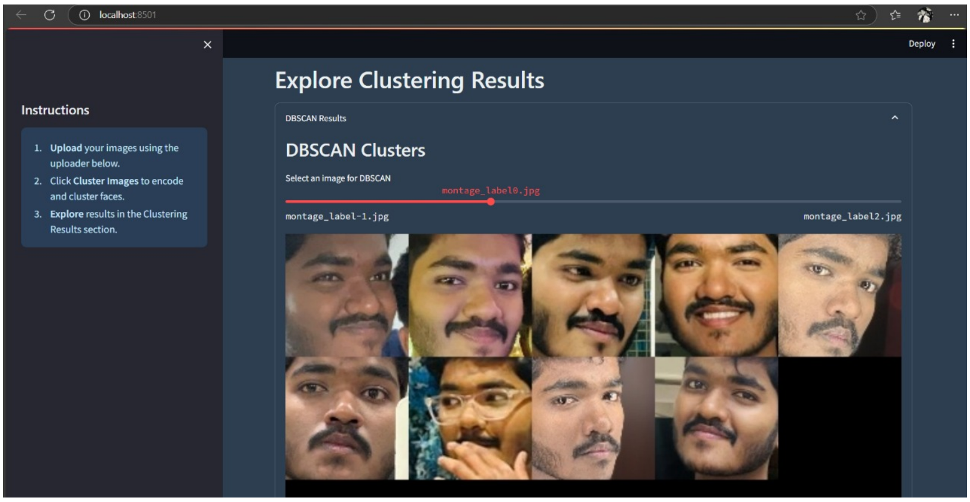
8 Contribution

Table 2: Team Member Contributions

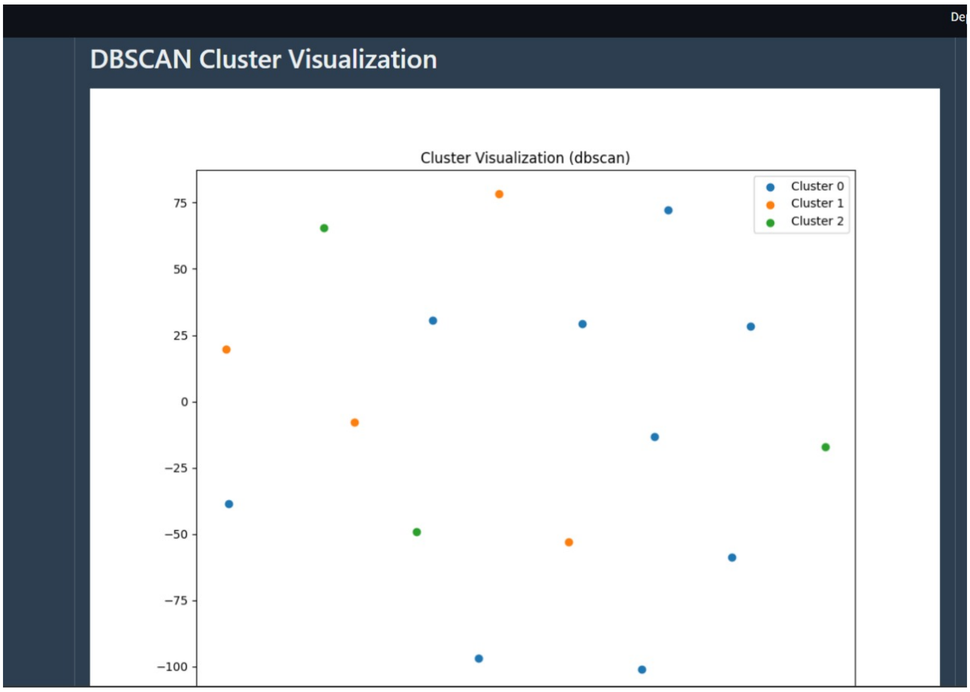
Member	Roll Number	Contribution Area	Algorithms / Modules
V. Sujay	B22CS063	Clustering Algorithms (CV-Based)	KMeans Clustering Agglomerative Clustering Web App Development & Integration
Pareen Shah	B22CS038	Clustering + Detection	DBSCAN Clustering MTCNN Detection Web App Development & UI Design
Aiswarya K	B22CS028	Classical Encoding Techniques	PCA (Principal Component Analysis) LDA (Linear Discriminant Analysis)
M. Suresh	B22CS031	Deep Learning Embedding Models	FaceNet Embedding Autoencoder-Based Face Encoding
Y. Nitin Manoj	B22CS066	Deep Learning Embedding Models	FaceNet Embedding Autoencoder-Based Face Encoding

9 Deployed Results

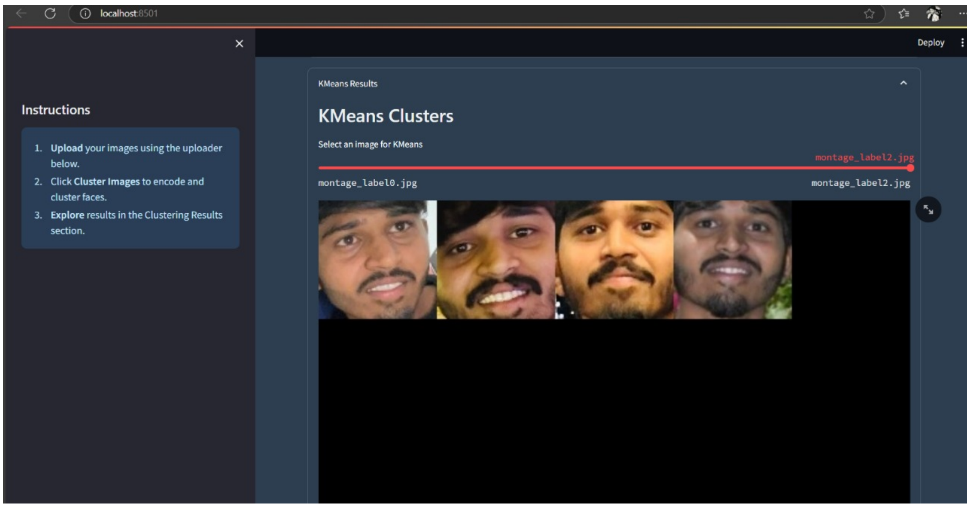




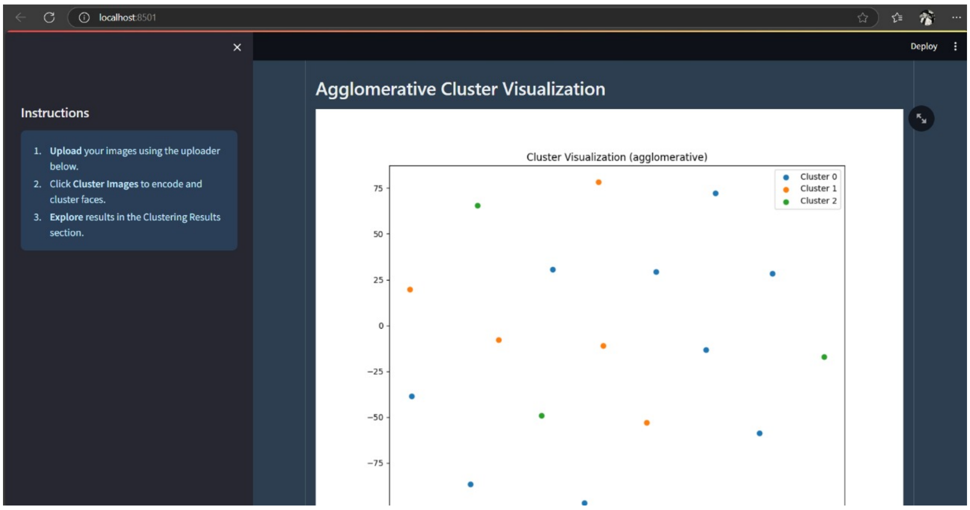
localhost:8501
Deploy
Instructions
1. Upload your images using the uploader below.
2. Click Cluster Images to encode and cluster faces.
3. Explore results in the Clustering Results section.
Explore Clustering Results
DBSCAN Results
DBSCAN Clusters
Select an image for DBSCAN
montage_label0.jpg
montage_label-1.jpg
montage_label2.jpg



DBSCAN Cluster Visualization
Cluster Visualization (dbscan)
Cluster 0
Cluster 1
Cluster 2
75
50
25
0
−25
−50
−75
−100



localhost:8501
Deploy
Instructions
1. Upload your images using the uploader below.
2. Click Cluster Images to encode and cluster faces.
3. Explore results in the Clustering Results section.
KMeans Results
KMeans Clusters
Select an image for KMeans
montage_label2.jpg
montage_label0.jpg
montage_label2.jpg


localhost:8501
Deploy
Instructions
1. Upload your images using the uploader below.
2. Click Cluster Images to encode and cluster faces.
3. Explore results in the Clustering Results section.
Agglomerative Cluster Visualization
Cluster Visualization (agglomerative)
Cluster 0
Cluster 1
Cluster 2
75
50
25
0
−25
−50
−75